

APACHE SPARK

Structured Streaming

A complete, in-depth guide: from readStream to watermarks

- Core concepts & API
- Transformations & checkpoints
- Output modes & triggers
- Event time, windows & watermarks
- Use cases & best practices

What Is Streaming?

The shift from collecting data first to reacting to it as it happens.

Definition

Streaming means processing data continuously as it arrives, instead of waiting until all the data has been collected. Each new record is handled the moment it shows up, rather than in one large, delayed run.

Why It Matters

- Decisions and dashboards reflect what's happening right now, not what happened hours ago
- Systems can react to events (fraud, outages, orders) within seconds of them occurring
- Data volume no longer has to accumulate before it becomes useful

The Core Idea

Traditional approach

Wait → collect a full dataset → then process it in one pass

Streaming approach

Process each record (or small micro-batch) the moment it arrives

What Is Structured Streaming?

Spark's stream-processing engine — the same DataFrame & Spark SQL APIs you already use for batch, applied to a continuous flow of data.

Structured Streaming = Spark DataFrame + continuous incoming data

Key Benefits

- Unified API — the exact same filter(), groupBy(), and join() calls used in batch jobs work unchanged on a stream
- Incremental execution — Spark only computes what's new since the last micro-batch, not the whole dataset again
- Built-in fault tolerance — checkpointing lets a query recover exactly where it left off after a failure
- Declarative — you describe the transformation once; Spark figures out how to run it continuously

The pattern

`readStream` → `transform (DataFrame ops)` → `writeStream`

```
df = (  
    spark.readStream  
        .format("json")  
        .schema(schema)  
        .load("/data/orders")  
)  
  
# same DataFrame API as batch  
result = df.filter(df.amount > 100)  
  
query = (  
    result.writeStream  
        .format("console")  
        .outputMode("append")  
        .option("checkpointLocation",  
                "/chk/orders")  
        .start()  
)  
  
query.awaitTermination()
```

Batch vs. Streaming

Same engine, same APIs — the difference is when and how often the code runs.

BATCH

Process data that already exists

read → transform → write

- Runs once, on a fixed, finite dataset
- Job finishes and the process exits
- Latency measured in minutes to hours
- Simpler to reason about and debug

```
df = spark.read.format("json")  
    .load("/data/orders")  
  
df.write.format("parquet")  
    .save("/out")
```

STREAMING

Process data continuously as it arrives

readStream → transform → writeStream

- Runs indefinitely against an unbounded stream
- Process stays alive, processing micro-batches
- Latency measured in seconds to minutes
- Needs checkpointing to be fault-tolerant

```
df = spark.readStream.format("json")  
    .load("/data/orders")  
  
df.writeStream.format("parquet")  
    .start()
```

Stateless vs. Stateful Transformations

Whether Spark needs to remember anything from records it already processed.

Stateless

Spark processes each incoming record without needing to remember information from previous records. Every row is transformed independently and immediately.

`select()`

`filter()`

`withColumn()`

`drop()`

`cast()`

`union()`

`map()`

Typical use: cleaning, filtering rows, renaming or casting columns, parsing a single record's fields.

Stateful

Spark must remember information from previous data — like a running total or a set of seen keys — to compute the current result correctly.

`groupBy().sum()`

`groupBy().count()`

`groupBy().agg()`

`window aggregation`

Typical use: running totals, deduplication, sessionization, and any windowed aggregation. This state lives in the checkpoint location.

Checkpointing

A location where Spark stores progress & state info for a streaming query — its recovery mechanism. If the query crashes or is restarted, Spark reads the checkpoint and resumes exactly where it left off, instead of starting over or reprocessing data.

1 Commits

Information about successfully committed micro-batches — which batches finished.

2 Offsets

Offset/progress information for the source — how far Spark has read.

3 Source

Information about the streaming source(s) — where it's reading data from.

4 State

Stateful operator state — what previous state Spark needs to remember.

```
query = (  
    df.writeStream  
        .format("parquet")  
        .option("path",  
                "/data/output")  
        .option("checkpointLocation",  
                "/data/checkpoint")  
        .start()  
)
```

Best practice: never reuse a checkpoint directory across two different queries — the offsets and state won't match the new logic, causing incorrect or failed restarts.

Output Modes

Tells Spark which rows should be written to the sink every time the streaming query produces an updated result.

Append

Only new rows that are finalized are written to the output. Spark never rewrites old output rows once they've been emitted.

BEST FOR

Stateless transformations; sinks like files, Kafka

Complete

The entire current result table is rewritten to the sink every time the result changes — not just the delta.

BEST FOR

Stateful aggregations with small result sets

Update

Only rows whose results changed since the previous trigger are written — a smaller, incremental delta of Complete mode.

BEST FOR

Stateful aggregations written to mutable sinks

Triggers & Trigger Types

A trigger determines when Spark should process the available data — when it should run the next micro-batch.

Trigger	Processing Style	Main Use
Default	As fast as possible	Normal streaming — lowest possible latency with no fixed schedule
ProcessingTime	Micro-batch at a fixed interval	Scheduled intervals — predictable cadence, e.g. every 10s
AvailableNow	Processes all currently available data, then stops	Incremental ETL — catch up, then exit cleanly
Once	Processes exactly one micro-batch, then stops	One-time or ad hoc processing runs
Continuous	Continuous, low-latency, record-at-a-time processing	Specialized use cases needing millisecond latency

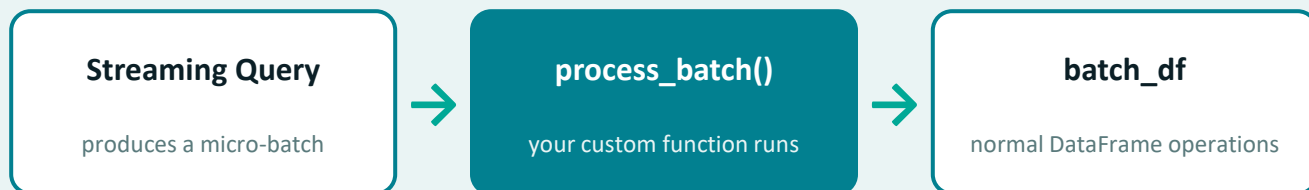
Example

```
df.writeStream
  .trigger(processingTime="10 seconds")
  .start()

df.writeStream
  .trigger(availableNow=True)
  .start()
```


foreachBatch()

Run custom code on each micro-batch produced by a streaming query, using normal Spark DataFrame operations — every time Spark creates a micro-batch, your function runs on that batch.



Common Use Cases

- Writing the same batch to multiple sinks (e.g. a database and a data lake)
- Using batch-only APIs, like a JDBC upsert, not natively supported by `writeStream`
- Reusing existing batch DataFrame logic inside a streaming pipeline

```
def process_batch(batch_df, batch_id):  
    # normal DataFrame operations  
    batch_df.write.jdbc(  
        url=db_url, table="orders", mode="append")  
    batch_df.show()  
  
query = df.writeStream.foreachBatch(process_batch).start()  
query.awaitTermination()
```

Event Time vs. Processing Time

Two different clocks matter in streaming — and they rarely agree exactly.

Event Time

The timestamp attached to the event by the producer/source — when the event actually happened, out in the real world.

Processing Time

The time when Spark actually processes the event — when the record reaches and is handled by the engine.

Why the gap matters: a mobile app might buffer events offline for 10 minutes before syncing — event time is 10:02, but Spark doesn't process it until 10:12. Aggregations based on processing time would put it in the wrong bucket.

Windowing by event time

```
result = (
  df
  .groupBy(
    window("event_time", "5 minutes")
  )
  .agg(count("*").alias("orders"))
)
```

Window Operations — Tumbling & Sliding

Windowing divides a continuous stream into time-based groups so calculations can run per group.

Tumbling Window

- Fixed duration
- No overlap between windows
- Each event belongs to exactly one window

```
window("event_time", "5 minutes")
```

Example: a 5-minute tumbling window creates non-overlapping buckets — [10:00–10:05), [10:05–10:10), [10:10–10:15) — each event falls into exactly one.

Sliding Window

- Fixed window duration
- A separate sliding interval
- Windows can overlap — one event can belong to several windows

```
window("event_time",  
      "10 minutes", "5 minutes")
```

Example: a 10-minute window sliding every 5 minutes creates [10:00–10:10), [10:05–10:15), [10:10–10:20) — each event contributes to 2 windows.

Window Operations — Session Windows

Groups events that belong to the same activity or session of a user — a variable-length window driven by behavior, not the clock.

How it works

- Groups events based on activity or inactivity, rather than a fixed clock interval
- A gap is the period of inactivity that closes a session — if the user is idle longer than the gap, the next event starts a new session
- Each user can have many sessions over time, and sessions can be very different lengths
- Ideal for measuring engagement, browsing behavior, or any activity with natural start/stop boundaries

```
result = df.groupBy(  
    "user_id",  
    session_window("timestamp",  
                   "10 minutes")  
) .count()
```

Worked example

With a 10-minute gap: events at 10:00, 10:04, and 10:09 all fall in one session (each within 10 min of the last). An event at 10:25 is more than 10 minutes after 10:09, so it starts a brand-new session.

Late Events & Watermarks

Late Event

An event that arrives at the streaming system after the time it actually happened — e.g. it happened at 10:02, but network delay means Spark receives it at 10:08.

Watermark

A limit on how late event-time data can be for a stateful streaming operation. It tells Spark how long to wait for late-arriving events before treating a window as final.

Why it matters

Watermarks let Spark decide when old event-time data is too late to keep affecting state — and when that old state can safely be cleaned up, so it doesn't grow forever.

```
result = (
    df
    .withWatermark("event_time",
                  "10 minutes")
    .groupBy(
        window("event_time",
              "5 minutes")
    )
    .agg(count("*").alias("event_count"))
)
```

Worked example

Event Time: when did the event actually happen?

Window: which time period does the event belong to?

Watermark: how far Spark has progressed in event time, and how much lateness it will tolerate.

With a 10-min watermark, a 5-min window ending 10:05 stays open for late data until event time passes 10:15 — then it finalizes and its state is dropped.

Common Use Cases

Where Structured Streaming shows up in real data platforms.

Real-Time Dashboards

Streaming aggregations (counts, sums, averages) feed live dashboards so metrics update within seconds instead of overnight.

Fraud & Anomaly Detection

Stateful transformations flag suspicious patterns — like rapid repeated transactions — the moment they happen.

Log & Event Pipelines

readStream ingests application or IoT logs continuously, cleans them, and writes to a data lake or warehouse.

ETL / Incremental Loading

AvailableNow or ProcessingTime triggers turn Structured Streaming into a scheduled, incremental ETL job.

Kafka Integration

Consuming from and producing to Kafka topics is a first-class source/sink, powering event-driven microservices.

Sessionization & Analytics

Session windows group user activity for engagement metrics, funnel analysis, and personalization.

Key Takeaways

- ✓ Structured Streaming reuses the DataFrame API — `readStream/writeStream` instead of `read/write` — so batch skills transfer directly.
- ✓ Stateless transformations process each record independently; stateful ones remember prior data (running totals, aggregations).
- ✓ Checkpoints store commits, offsets, source & state so a query can recover exactly where it left off after a failure.
- ✓ Choose the output mode — append, complete, or update — to match stateless vs. stateful logic and your sink's capabilities.
- ✓ Triggers control cadence, from continuous low-latency processing to one-time `Once/AvailableNow` batches.
- ✓ Windowing (tumbling, sliding, session) groups events by event time, not processing time — the two clocks can diverge.
- ✓ Watermarks bound how late data can be, letting Spark finalize windows and drop old state instead of growing forever.